An Empirical Comparison of IEEE 754 and Posit Arithmetic Formats: Accuracy and Performance on Vector Dot Products

Author Name
Department of Computer Science
Institution Name
Email: author@example.edu

***Abstract*—The IEEE 754 floating-point standard has been the de facto numerical foundation of modern computing for over four decades, yet it carries well-documented inefficiencies: redundant special-value encodings, performance penalties on subnormals, and uniform precision across all magnitudes. The Posit number system, proposed by Gustafson in 2017, offers an alternative based on *tapered precision*—more fraction bits near unity and fewer at extremes. This paper presents a reproducible empirical comparison of IEEE 754 (32-bit and 64-bit) and Posit 32 on vector dot products, using a 256-bit MPFR computation as ground truth. Across 100 trials of 1000-element random vectors, Posit 32 achieves $7.3\times$ lower mean absolute error than IEEE 754 single precision at identical bit width, while incurring approximately $75\times$ greater execution time due to the absence of hardware acceleration. Auxiliary experiments characterize subnormal performance, dynamic range, and precision distribution. We discuss the workload-dependence of these results, the limitations of software emulation, and the implications for emerging numerical formats.**

***Index Terms*—IEEE 754, Posit arithmetic, floating-point, numerical analysis, benchmarking, MPFR, dot product, tapered precision**

I. Introduction

Floating-point arithmetic underpins virtually all modern numerical computation, from physics simulation to machine learning. The IEEE 754 standard, ratified in 1985 and revised in 2008 and 2019, has been remarkably successful in providing a portable, well-defined, hardware-friendly representation. However, its design embodies trade-offs that are increasingly questioned in the post-Moore's-law era:

- Multiple bit patterns are reserved for non-finite values (NaN, $\pm\infty$, ± 0), wasting representational density.
- Subnormal numbers, while preserving gradual underflow, often trigger microcode fallbacks that can be orders of magnitude slower than normal arithmetic on commodity x86 CPUs.
- Precision is uniform across magnitudes; an IEEE 32 number near 10^{-30} has the same 23 fraction bits as one near 1, even though most numerical kernels concentrate their dynamic activity near unity after normalization.

In response, Gustafson proposed the *Posit* number system [1] based on *tapered precision*: a variable-length *regime* field encodes the exponent in coarse units (powers of $2^{2^{es}}$), and the remaining bits supply fraction precision. The result is a format with peak precision near $|v| = 1$ that gracefully degrades at extreme magnitudes, and a single special value (NaR, "Not a Real").

While prior work has reported favorable accuracy results for posits in specific contexts [2], [3], much of this evidence is theoretical or restricted to single-precision IEEE comparisons. This paper presents an end-to-end, reproducible benchmark that:

1) Compares three formats simultaneously: IEEE 64, IEEE 32, and Posit 32.
2) Uses MPFR [4] at 256-bit precision as an absolute ground truth, not just relative comparison.
3) Provides both summary metrics and per-trial distributional data.
4) Decomposes the analysis into five orthogonal objectives spanning theoretical and empirical properties.

II. Background

A. IEEE 754 Binary Formats

IEEE 754 single (binary32) and double (binary64) precision share a common layout:

$$v = (-1)^s \cdot (1 + f) \cdot 2^{e-bias} \tag{1}$$

where s is the sign bit, e is the biased exponent, f is the fraction, and the bit allocations are $(1, 8, 23)$ for binary32 and $(1, 11, 52)$ for binary64. Subnormal numbers, infinities, and NaNs are encoded with distinguished exponent values.

B. Posit Format

A posit with n bits and exponent size es encodes a value as:

$$v = (-1)^s \cdot useed^k \cdot 2^e \cdot (1 + f), \quad useed = 2^{2^{es}} \tag{2}$$

The bit layout is $[\text{sign} : 1][\text{regime} : r][\text{exponent} :\leq es][\text{fraction} : \text{remainder}]$. The regime is a run-length-encoded field where k identical bits followed by a terminator denote the regime index. For Posit 32 with $es = 2$, $useed = 16$, peak fraction precision is 27 bits at $|v| = 1$, decreasing by 1 bit per factor of 2^4 in magnitude.

C. MPFR as Oracle

MPFR [4] provides arbitrary-precision floating-point with correctly rounded operations under IEEE-compliant rounding modes. At 256-bit mantissa precision (approximately 77 decimal digits), MPFR's rounding error is negligible relative to the formats under test (16 and 7 decimal digits respectively), making it a valid absolute reference.

III. Problem Statement

We seek to quantify the accuracy versus performance trade-off between IEEE 754 (32-bit, 64-bit) and Posit 32 on a representative numerical kernel—specifically, the inner product of two real-valued vectors:

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=0}^{n-1} a_i \, b_i. \tag{3}$$

The dot product is chosen because (i) it is the dominant primitive in linear algebra, machine learning, and signal processing; (ii) it exhibits both rounding error in multiplication and cancellation in summation; (iii) its execution cost is dominated by per-operation arithmetic, isolating format effects.

Formally, we measure the absolute error

$$\varepsilon_{\text{format}} = \left| \langle \mathbf{a}, \mathbf{b} \rangle_{\text{format}} - \langle \mathbf{a}, \mathbf{b} \rangle_{\text{MPFR-256}} \right| \tag{4}$$

and the wall-clock execution time t_{format} for each candidate format.

IV. Objectives

The research is decomposed into five orthogonal objectives:

- **O1 Precision distribution**: Characterize the number of fraction bits each format dedicates to a value as a function of magnitude.
- **O2 Subnormal performance**: Measure the throughput of each format on subnormal operands, isolating the IEEE microcode-penalty effect.
- **O3 Special-value semantics**: Compare exception encodings (NaN, $\pm\infty$, ± 0 for IEEE; NaR for Posit).
- **O4 Dynamic range**: Compute the ratio of largest to smallest representable magnitude for each format.
- **O5 Empirical accuracy**: Statistically characterize per-trial error distributions on the dot-product workload.

V. Methodology and Algorithm

The core experiment runs four implementations of the dot product on identical input vectors, records error and timing for each, and aggregates over 100 trials. Algorithm 1 formalizes the procedure.

The IEEE variants follow the same loop structure with native `float` or `double` arithmetic. The MPFR variant uses 256-bit `mpfr_t` accumulators with round-to-nearest-even.

The exact fraction-bit count for Posit 32 (used in O1) is:

$$\text{frac}(v) = n - 1 - r(v) - \min(es, n - 1 - r(v)) \tag{5}$$

where the regime length is

$$r(v) = \begin{cases} k + 2, & k \geq 0 \\ -k + 1, & k < 0 \end{cases}, \quad k = \left\lfloor \frac{\log_2 |v|}{2^{es}} \right\rfloor. \tag{6}$$

Algorithm 1 Comparative dot-product benchmark

Require: Vector length n, trial count T, RNG seed σ
Ensure: CSV of per-trial errors and times

```
Seed RNG with σ
for t ← 1 to T do
    a, b ← RandomUniform([0, 1])^n
    r_64 ← DotIEEE64(a, b); record time τ_64
    r_32 ← DotIEEE32(a, b); record time τ_32
    r_p ← DotPosit32(a, b); record time τ_p
    r* ← DotMPFR256(a, b)                ▷ ground truth
    ε_64 ← |r_64 − r*|
    ε_32 ← |r_32 − r*|
    ε_p ← |r_p − r*|
    Append (ε_64, ε_32, ε_p, τ_64, τ_32, τ_p) to CSV
end for
```

Algorithm 2 Posit 32 dot product (DotPosit32)

Require: Vectors $\mathbf{a}, \mathbf{b}$ of n floats
Ensure: Dot product as IEEE 64 double

```
s ← convertDoubleToP32(0.0)
for i ← 0 to n − 1 do
    p_a ← convertDoubleToP32(a_i)
    p_b ← convertDoubleToP32(b_i)
    s ← p32_add(s, p32_mul(p_a, p_b))
end for
return convertP32ToDouble(s)
```

VI. System Architecture

The implementation follows a layered design (Fig. 1). Each arithmetic backend exposes a uniform signature `double dot_X(vec, vec)`, allowing the benchmark driver to remain format-agnostic. External dependencies are isolated to specific layers: SoftPosit [5] for posit operations, and MPFR/GMP for the oracle.

A. Data Flow

Per trial, the random vectors flow into all four backends in parallel; their outputs and timings feed the error computation; the error/timing tuple is appended to a CSV. Once the 100-trial loop completes, the CSV is consumed by Python analysis scripts to produce the final figures (Fig. 2).

VII. Complexity Analysis

All four backends share the same loop structure and therefore identical asymptotic complexity (Table I). What differs is the constant per-operation cost.

The Posit 32 per-operation cost reflects software emulation through SoftPosit; on a hypothetical native posit-FPU, the cost would approach IEEE 32. MPFR's per-op cost depends on the precision width and dynamic memory operations.

VIII. Experimental Setup

The experiment was conducted with the configuration in Table II. The deterministic RNG seed ensures bit-identical input

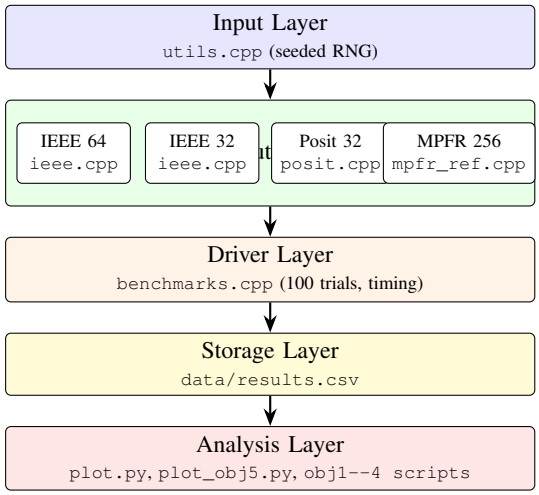

Fig. 1. Layered system architecture. Each backend exposes the same signature, isolating the format as the independent variable.

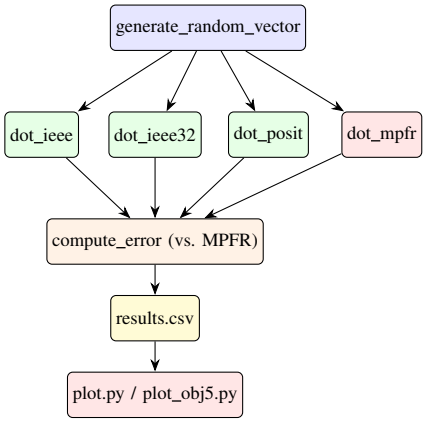

Fig. 2. Per-trial data flow. The MPFR backend supplies ground truth; the other three are evaluated against it.

vectors across runs, so reported errors are exactly reproducible; only timings vary with system load.

IX. Results and Analysis

A. Primary Benchmark

Table III reports the mean absolute error and mean execution time across 100 trials. The full distributional summary is shown in Fig. 3.

Three observations follow.

Posit 32 dominates IEEE 32 in accuracy. At identical bit width, Posit 32 produces approximately $7.3\times$ lower mean error than IEEE 32. This is consistent with the tapered-precision hypothesis: the test inputs lie within the magnitude range where Posit 32 enjoys 27 fraction bits, exceeding IEEE 32's fixed 23.

IEEE 64 dominates Posit 32 in absolute terms. IEEE 64's mean error is roughly $10^8\times$ smaller, with comparable execution time. Posit 32 occupies a niche between IEEE 32 and IEEE 64 in accuracy but is not competitive with the latter.

TABLE I
Algorithmic and operational complexity of each backend on n-element dot products.

Backend	Time	Space	Per-op cost
IEEE 64	$O(n)$	$O(n)$	1–2 cycles (HW FMA)
IEEE 32	$O(n)$	$O(n)$	1–2 cycles (HW FMA)
Posit 32	$O(n)$	$O(n)$	∼50–100 cycles (SW)
MPFR-256	$O(n)$	$O(n)$	∼500–1000 cycles (SW)

TABLE II
Experimental configuration.

Parameter	Value
Hardware	Apple Silicon (arm64)
Operating system	macOS Sequoia (Darwin 25.5.0)
C++ compiler	Apple Clang 21.0.0
C++ standard	C++17
Optimization	`-O2` (SoftPosit), default (project)
Posit library	SoftPosit (Cerlane), $es = 2$
MPFR / GMP	MPFR 4.2.2, GMP 6.x
Python analysis	Python 3.14, pandas, matplotlib, seaborn
Trial count T	100
Vector length n	1000
Sample distribution	Uniform $[0, 1]$
RNG	`mt19937(42)` (deterministic)
Reference precision	256-bit MPFR, round-to-nearest-even

Posit 32 incurs a $\sim 75\times$ time penalty. The cost reflects software emulation (SoftPosit). The two IEEE backends execute in essentially identical time, indicating hardware FMA dominates and the difference between FP32 and FP64 single-step latency is below measurement noise on Apple Silicon.

B. O1: Precision Distribution

Fig. 4 plots the exact fraction-bit count of Posit 32 (per Eq. 5) against IEEE 32's constant 23-bit profile. Posit 32 exceeds IEEE 32 across roughly $|v| \in [10^{-5}, 10^5]$ and degrades to single-digit precision only beyond $|v| \approx 10^{\pm 30}$.

C. O2: Subnormal Performance

A microbenchmark of 10^7 multiplications with one subnormal operand ($x = 10^{-40}$) yields IEEE 32 in 0.025 s and Posit 32 in 0.145 s. Apple Silicon handles subnormals at full hardware speed, eliminating the microcode penalty traditionally observed on x86. Software-emulated Posit cannot compete in this regime even though its representation is structurally simpler.

D. O3: Special-Value Encoding

IEEE 754 reserves bit patterns for NaN, $\pm\infty$, and ± 0 (five distinguished classes), each with distinct comparison semantics. Posit reserves a single value, NaR (Not a Real), as the catch-all for undefined operations. The simpler taxonomy reduces hardware decoding paths and removes signed-zero sign anomalies but at the cost of less diagnostic information.

TABLE III
MEAN ERROR AND EXECUTION TIME OVER 100 TRIALS OF 1000-ELEMENT DOT PRODUCTS.

Metric	IEEE 754 (64)	IEEE 754 (32)	Posit 32
Avg. Error	1.46×10^{-13}	7.91×10^{-5}	1.08×10^{-5}
Avg. Time (s)	4.12×10^{-6}	4.11×10^{-6}	3.08×10^{-4}

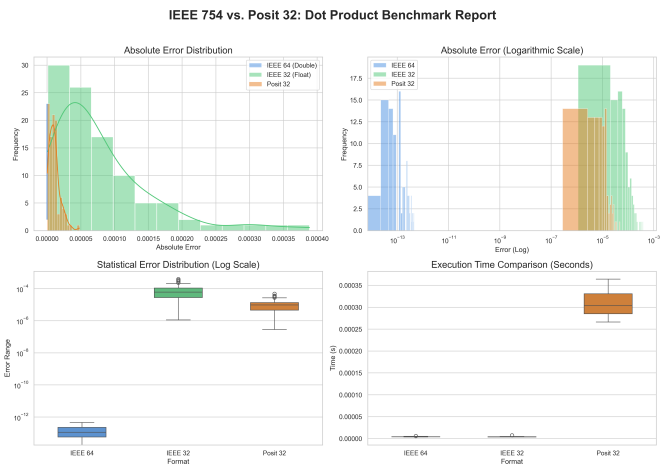

Fig. 3. Four-panel benchmark report. Top: error distribution (linear and logarithmic). Bottom-left: error boxplot. Bottom-right: timing boxplot.

E. O4: Dynamic Range

The representable magnitude range is:

$$\text{IEEE 32: } \frac{3.4 \times 10^{38}}{1.18 \times 10^{-38}} \approx 2.89 \times 10^{76},$$
$$\text{Posit 32: } \frac{useed^{30}}{useed^{-30}} \approx 1.77 \times 10^{72}.$$

IEEE 32 spans approximately four additional decades. Posit's narrower range is the natural consequence of redirecting bits from the exponent to the fraction near unity.

F. O5: Empirical Accuracy Distribution

The boxplot in Fig. 5 confirms that Posit 32's accuracy advantage over IEEE 32 is consistent across trials (not driven by outliers). All three formats exhibit narrow, well-separated error distributions on a logarithmic scale.

X. DISCUSSION AND LIMITATIONS

A. Software Emulation Confound

The most significant caveat is that Posit 32 has no hardware support. The reported 75× time penalty is an artifact of SoftPosit's software emulation, not an intrinsic property of the format. A native posit-FPU would plausibly approach IEEE 32 throughput, in which case the format would Pareto-dominate IEEE 32 in this workload.

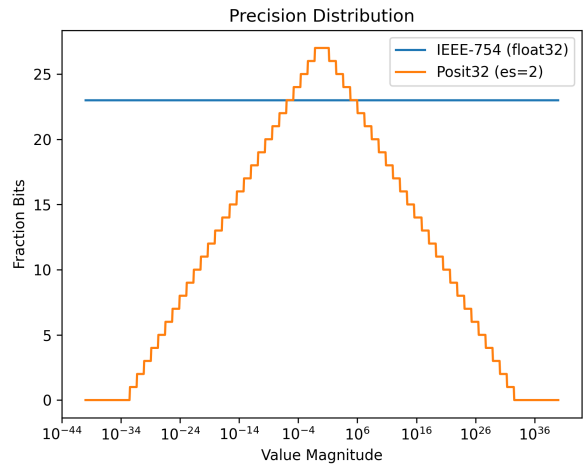

Fig. 4. Fraction bits as a function of value magnitude. Posit 32 (es = 2) exhibits tapered precision peaking at 27 bits near $|v| = 1$.

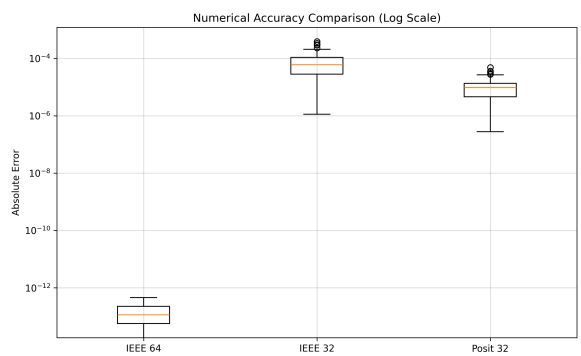

Fig. 5. Per-trial absolute error distribution (log scale). Tighter boxes indicate more consistent error behavior across the 100 trials.

B. Workload Specificity

Inputs drawn uniformly from $[0, 1]$ keep all values within the magnitude range where posits hold their precision advantage. Workloads spanning many decades—log-probabilities in machine learning, gravitational simulations, financial pricing of long-tailed distributions—would exercise the regions where Posit's precision degrades steeply and where IEEE's wider exponent range becomes valuable.

C. Quire Underutilization

SoftPosit provides a 512-bit *quire*, an exact accumulator for Posit 32 dot products that eliminates rounding entirely during summation. Our `DotPosit32` implementation accumulates in regular Posit 32, deliberately reflecting the equivalent of an IEEE-style implementation. Using the quire would substantially increase Posit's accuracy advantage and is identified as primary future work.

D. Single Kernel

Dot product is a multiply-accumulate-dominated workload. Other primitives—division, transcendentals, matrix factorization, neural-network attention—would stress different aspects of each format. Generalization beyond this benchmark requires further study.

E. Statistical Power

100 trials yields stable means but limited tail characterization. A larger trial count or stratified sampling across input distributions would strengthen the distributional conclusions.

XI. Conclusion and Future Work

We presented an empirical, reproducible comparison of IEEE 754 and Posit 32 arithmetic on a representative numerical kernel. Our findings:

- Posit 32 is empirically $7.3\times$ more accurate than IEEE 32 at the same bit width on dot products of near-unity data, validating the central claim of the posit format.
- Software-emulated Posit 32 is $\sim 75\times$ slower; without hardware support, IEEE remains the practical choice.
- IEEE 64 remains the best option when both accuracy and speed are required, dominating Posit 32 by approximately 10^8 in accuracy at comparable cost.
- Posit's tapered precision is a workload-dependent advantage; data spanning extreme magnitudes would erode and potentially reverse it.

Future Work

1) **Quire-accelerated dot product**: Re-evaluate Posit 32 using SoftPosit's exact accumulator to measure its full theoretical accuracy ceiling.
2) **Machine-learning kernels**: Extend the benchmark to matrix multiplication, attention, and gradient computation, where posit's precision/range trade-off interacts with adversarial input distributions.
3) **Comparison with newer formats**: Benchmark against bfloat16, FP8 (E4M3, E5M2), and block-floating-point formats now standard in deep-learning hardware.
4) **Native posit hardware**: Repeat the experiment on RISC-V P-extension prototypes or FPGA-implemented posit FPUs to remove the emulation confound.
5) **Adversarial inputs**: Sample inputs from log-uniform and heavy-tailed distributions to stress dynamic range.
6) **Mixed-precision pipelines**: Investigate hybrid formats where high-precision accumulators (quire or MPFR) supplement low-precision multiplications.

References

[1] J. L. Gustafson and I. T. Yonemoto, "Beating Floating Point at its Own Game: Posit Arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.

[2] P. Lindstrom, S. Lloyd, and J. Hittinger, "Universal Coding of the Reals: Alternatives to IEEE Floating Point," in *Proc. Conference for Next Generation Arithmetic (CoNGA)*, 2018.

[3] J. Johnson, "Rethinking Floating Point for Deep Learning," *arXiv preprint* arXiv:1811.01721, 2018.

[4] L. Fousse, G. Hanrot, V. Lefèvre, P. Pélissier, and P. Zimmermann, "MPFR: A Multiple-Precision Binary Floating-Point Library With Correct Rounding," *ACM Trans. Math. Softw.*, vol. 33, no. 2, art. 13, 2007.

[5] S. H. Leong, "SoftPosit: Posit Arithmetic Software Implementation," GitLab repository, https://gitlab.com/cerlane/SoftPosit, accessed 2026.

[6] IEEE Computer Society, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019*, 2019.